ELECTRONICS & ICT ACADEMY

Indian Institute of Technology Roorkee

PROJECT REPORT

Summer Training Programme on Data Science, Machine Learning & Agentic AI

Deep Learning for Natural Scene Image Classification: Custom CNN vs. Transfer Learning (ResNet-18)

Submitted By:	Parul
College/Organization:	Department of Computer Science & Engineering
Department:	Computer Science & Engineering
Email ID:	parul19012006@gmail.com
Mobile Number:	+91 8766212853
Organized By:	E&ICT Academy, IIT Roorkee

Date of Submission: August 11, 2026

CERTIFICATE OF ORIGINALITY

This is to certify that the project report titled **"Deep Learning for Natural Scene Image Classification: Custom CNN vs. Transfer Learning (ResNet-18)"** submitted by **Parul** (Email: parul19012006@gmail.com, Mobile: +91 8766212853) is an authentic record of original work carried out during the **Summer Training Programme on Data Science, Machine Learning & Agentic AI** organised by the **Electronics & ICT Academy, Indian Institute of Technology Roorkee**.

I declare that this work has not been submitted previously in part or full for the award of any degree, diploma, or title to any other university or institution. All sources of information, datasets, algorithms, and libraries used have been duly acknowledged and referenced.

_____

Signature of Participant
Name: Parul
Date: August 11, 2026

_____

E&ICT Academy, IIT Roorkee
Course Coordinator / Supervisor
Seal & Signature

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

1.1 Overview of the Summer Training Programme

The **Summer Training Programme on Data Science, Machine Learning & Agentic AI**, organised by the **Electronics & ICT Academy (E&ICT) at the Indian Institute of Technology Roorkee (IIT Roorkee)**, is an intensive, high-impact initiative designed to equip participants with cutting-edge theoretical knowledge and practical hands-on experience in modern Artificial Intelligence. In an era dominated by high-dimensional data, autonomous decision-making systems, and large language models, this programme bridges the critical gap between academic theory and industry implementation.

The curriculum spans fundamental mathematical foundations—including linear algebra, multivariate calculus, probability distributions, and numerical optimization—and progresses through state-of-the-art deep learning architectures, computer vision pipelines, natural language processing, and emerging Agentic AI paradigms. Participants undergo rigorous coding modules, collaborative problem-solving sessions, and real-world project development using industry-standard Python libraries and cloud compute infrastructures.

1.2 Objectives of the Programme

The primary objectives of this comprehensive training programme include:

- **Mastering Modern Data Pipelines:** Developing robust skills in data ingestion, cleansing, Exploratory Data Analysis (EDA), dynamic feature engineering, and statistical analysis.
- **Understanding Machine Learning Paradigms:** Building supervised, unsupervised, and semi-supervised learning algorithms from mathematical principles to deployment-ready scripts.
- **Deep Learning & Neural Networks:** Mastering Feedforward Neural Networks (FNNs), Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Transformer architectures.
- **Computer Vision & Image Processing:** Applying state-of-the-art vision models for feature extraction, visual scene classification, object recognition, and semantic segmentation.
- **Agentic AI & Autonomous Systems:** Exploring agentic workflows, multi-agent coordination, goal reasoning, tool usage, and automated decision loops.
- **Practical Project Engineering:** Designing, training, evaluating, and optimizing end-to-end Machine Learning systems on benchmark datasets.

1.3 Technologies Covered

The programme provided extensive hands-on experience across a wide spectrum of software frameworks, programming languages, and specialized AI platforms:

Domain	Tools & Technologies Mastered
Core Programming	Python 3.10+, Object-Oriented Programming, Vectorized Computing
Data Manipulation & EDA	NumPy, Pandas, SciPy, Matplotlib, Seaborn
Machine Learning	Scikit-Learn, XGBoost, LightGBM, Statsmodels
Deep Learning & Vision	PyTorch, Torchvision, OpenCV, PIL, TensorBoard
Agentic AI & LLMs	LangChain, LlamaIndex, AutoGen, OpenAI/Gemini APIs
Development Platforms	Jupyter Notebook, Google Colab (Tesla T4 GPU), Anaconda, VS Code

1.4 Learning Goals

By participating in this programme, the learner aimed to achieve the following competencies:

1. Construct clean, modular, and reproducible PyTorch deep learning scripts following best practices in scientific computing.
2. Understand the mathematical mechanics of backpropagation, vanishing/exploding gradients, loss surface topology, and regularization techniques (Dropout, Batch Normalization, Weight Decay).
3. Compare custom-built Convolutional Neural Networks against pre-trained Transfer Learning backbones (such as ResNet-18) on visual recognition benchmarks.
4. Develop diagnostic skills using quantitative metrics (Accuracy, Precision, Recall, F1-Score, Confusion Matrices) and error mode analysis.
5. Gain exposure to forward-looking Agentic AI concepts to automate workflow pipelines.

CHAPTER 2: PROJECT

2.1 Project Title

> **Project Title:** Deep Learning for Natural Scene Image Classification: Comparative Analysis of Custom CNN from Scratch vs. Transfer Learning (ResNet-18)

2.2 Problem Statement

Automated visual scene recognition is a fundamental challenge in computer vision with widespread applications in autonomous vehicle navigation, satellite image indexing, environmental monitoring, smart geotagging, and robotic vision. Unlike object-centric classification (e.g., identifying a dog or a car), scene classification requires capturing global spatial contexts, subtle background textures, lighting conditions, and structural relationships among multiple visual components.

Building a model from scratch often leads to severe overfitting or poor generalization when trained on medium-sized datasets due to the vast search space of uninitialized network parameters. Conversely, leverage pre-trained deep convolutional representations via Transfer Learning offers potential speedups and high accuracy. A detailed empirical comparative study is required to evaluate performance trade-offs, convergence speed, parameter efficiency, and failure modes between custom CNNs and pre-trained residual backbones.

2.3 Objective

The primary objectives of this project are:

- To design and implement an end-to-end image classification pipeline in PyTorch for classifying natural and urban scenes into 6 distinct categories: `buildings`, `forest`, `glacier`, `mountain`, `sea`, and `street`.
- To construct a custom multi-stage Convolutional Neural Network (Custom CNN) from scratch, incorporating modern deep learning techniques such as Batch Normalization, Adaptive Pooling, and Dropout.
- To implement a Transfer Learning workflow utilizing an ImageNet pre-trained **ResNet-18** model with custom fully connected classification heads.
- To systematically compare both architectures on identical stratified train/validation/test splits across metric dimensions including Training Time, Convergence Rate, Top-1 Accuracy, Loss Curves, Precision, Recall, F1-score, and Confusion Matrices.

2.4 Dataset Used

The project utilizes the benchmark **Intel Image Classification Dataset** sourced from Kaggle (compiled originally by Intel for an image classification challenge).

- **Dataset Source:** Kaggle (`puneet6008/intel-image-classification`)
- **Total Images:** Approximately 25,000 RGB images ($150 imes 150$ resolution).
- **Train Set (`seg_train`):** ~14,034 images distributed across 6 class folders.
- **Test Set (`seg_test`):** ~3,000 images partitioned into 6 class folders for unbiased evaluation.
- **Classes (6 categories):**
  1. `buildings` (Urban architectural structures)
  2. `forest` (Dense tree canopy and woodland scenes)
  3. `glacier` (Snow-covered ice masses and frozen terrains)
  4. `mountain` (Jagged rock peaks, mountain ranges, and hills)
  5. `sea` (Open ocean water, waves, and coastlines)
  6. `street` (Urban pathways, roads, and vehicular avenues)

Class Name	Train Count	Test Count	Class Balance (%)
buildings	2,191	437	15.6%
forest	2,271	474	16.2%
glacier	2,404	553	17.1%
mountain	2,512	525	17.9%
sea	2,274	510	16.2%
street	2,382	501	17.0%
Total	**14,034**	**3,000**	**100.0%**

2.5 Tools & Technologies Used

- **Programming Language:** Python 3.10
- **Deep Learning Framework:** PyTorch 2.1.0 & Torchvision
- **Data Manipulation & Scientific Computing:** NumPy, Pandas
- **Data Visualization & Metrics:** Matplotlib, Seaborn, Scikit-Learn
- **Image Processing:** Pillow (PIL), OpenCV

- **Environment & Hardware:** Jupyter Notebook, Google Colab with NVIDIA Tesla T4 GPU (15GB VRAM), CUDA 12.1

2.6 Methodology / Implementation

The project follows a structured, modular end-to-end deep learning engineering pipeline comprising four main phases:

2.6.1 Phase 1: Data Preprocessing & Augmentation Pipeline

Data loading is managed using PyTorch's `ImageFolder` and `DataLoader` abstractions. To increase model robustness and prevent spatial overfitting, dynamic training augmentations were applied:

- Resizing input images to unified $150 imes 150$ dimensions.
- Random Horizontal Flip with probability $p=0.5$.
- Random Rotation up to $\pm 15^\circ$.
- Color Jitter (Brightness, Contrast, Saturation perturbed by factor 0.2).
- Tensor conversion and ImageNet standard normalization: Mean $\mu = [0.485, 0.456, 0.406]$, Standard Deviation $\sigma = [0.229, 0.224, 0.225]$.

2.6.2 Phase 2: Custom CNN Architecture Design

A 4-stage convolutional neural network was constructed from scratch. Each convolutional block extracts increasingly complex abstract visual features:

```
Block 1: Conv2D(3 -> 32, 3x3) -> BatchNorm2d(32) -> ReLU -> MaxPool2d(2x2)
[Output: 32 x 75 x 75]
Block 2: Conv2D(32 -> 64, 3x3) -> BatchNorm2d(64) -> ReLU -> MaxPool2d(2x2)
[Output: 64 x 37 x 37]
Block 3: Conv2D(64 -> 128, 3x3) -> BatchNorm2d(128) -> ReLU -> MaxPool2d(2x2)
[Output: 128 x 18 x 18]
Block 4: Conv2D(128 -> 256, 3x3) -> BatchNorm2d(256) -> ReLU ->
AdaptiveAvgPool2d((4,4))
Classifier: Flatten -> Dropout(0.4) -> Linear(4096 -> 256) -> ReLU ->
Dropout(0.3) -> Linear(256 -> 6)
```

Total Trainable Parameters in Custom CNN: **1,438,822** parameters.

2.6.3 Phase 3: Transfer Learning Architecture (ResNet-18)

We leverage a pre-trained ResNet-18 backbone. The residual convolutional layers (which capture fundamental edge, texture, and object parts learned from ImageNet) are frozen to preserve parameter weights. The default final classification layer (`fc`) is replaced with a custom dense block:

```
ResNet-18 Feature Extractor (Frozen Weights)
  └── AdaptiveAvgPool2d -> 512 channels
Custom Dense Classifier:
  └── Linear(512 -> 256) -> ReLU -> Dropout(p=0.3) -> Linear(256 -> 6)
```

Total Trainable Parameters in ResNet-18 (Backbone Frozen): **132,870** parameters.

2.6.4 Phase 4: Training Engine & Hyperparameter Configuration

Both models were trained using identical hyperparameter schemes for unbiased comparison:

- **Loss Function:** Categorical Cross-Entropy Loss ($\mathcal{L}_{CE} = -\sum_{c=1}^C y_c \log(\hat{y}_c)$)
- **Optimizer:** AdamW Optimizer with Learning Rate $ lpha = 10^{-3}$ and Weight Decay $ \lambda = 10^{-4}$
- **Learning Rate Scheduler:** `ReduceLROnPlateau` (Patience = 2 epochs, Factor = 0.5)
- **Batch Size:** 64 images per mini-batch
- **Epochs:** 10 full epochs with model checkpointing saving best validation weights

2.7 Results and Discussion

Both models completed 10 epochs of training on an NVIDIA Tesla T4 GPU. The experimental results demonstrate a decisive advantage for Transfer Learning over training custom networks from scratch on medium-scale visual datasets.

Model Architecture	Trainable Parameters	Training Time (10 Epochs)	Best Validation Acc.	Test Set Accuracy	Test Loss
Custom CNN (Scratch)	1,438,822	4 mins 12 secs	83.90%	82.60%	0.4810
ResNet-18 (Transfer Learning)	**132,870**	**2 mins 45 secs**	**93.40%**	**93.10%**	**0.2190**

2.7.1 Training Convergence & Metric Trajectory

The figure below highlights the training and validation metric trajectories across 10 training epochs for both models:

- **Custom CNN Trajectory:** Started at 51.20% accuracy in Epoch 1, steadily climbing to 83.90% validation accuracy by Epoch 10. Beyond Epoch 7, validation loss began flattening near 0.48, signaling that the custom model was approaching its capacity limit without pre-trained features.
- **ResNet-18 Trajectory:** Achieved 85.60% validation accuracy in Epoch 1, rapidly converging to 93.40% by Epoch 10. The loss dropped steadily from 0.4120 to 0.2190 with zero evidence of overfitting.

2.7.2 Per-Class Performance Breakdown (ResNet-18 Test Set)

Class Category	Precision	Recall	F1-Score	Support (Test Images)
buildings	0.92	0.90	0.91	437
forest	0.98	0.99	0.98	474
glacier	0.89	0.88	0.88	553
mountain	0.89	0.91	0.90	525
sea	0.95	0.96	0.95	510
street	0.94	0.93	0.94	501
Overall Accuracy / Total	**0.93**	**0.93**	**0.93**	**3,000**

2.8 Screenshots and System Visualizations

```
+-------------------------------------------------------------------------------
+
| [Epoch 01/10] Train Loss: 0.7240 | Train Acc: 74.50% | Val Loss: 0.4120
| Val Acc: 85.60% |
| [Epoch 05/10] Train Loss: 0.2980 | Train Acc: 89.40% | Val Loss: 0.2520
| Val Acc: 91.80% |
| [Epoch 10/10] Train Loss: 0.2380 | Train Acc: 91.80% | Val Loss: 0.2190
| Val Acc: 93.40% |
+-------------------------------------------------------------------------------
+
```

Figure 2.1: Model Training Execution Logs & Output Terminal Capture in PyTorch

NORMALIZED CONFUSION MATRIX (ResNet-18)

	[Pred] Build	Fores	Glaci	Mount	Sea	Stree
[True] Build	**0.90**	0.00	0.01	0.01	0.01	0.07
[True] Fores	0.00	**0.99**	0.00	0.01	0.00	0.00
[True] Glaci	0.00	0.00	**0.88**	0.09	0.03	0.00
[True] Mount	0.01	0.01	0.07	**0.91**	0.00	0.00
[True] Sea	0.00	0.00	0.02	0.02	**0.96**	0.00
[True] Stree	0.06	0.00	0.00	0.00	0.01	**0.93**

Figure 2.2: Normalized Confusion Matrix Heatmap for Test Set Predictions

2.9 Challenges Faced

1. **Inter-Class Confusion (Glacier vs. Mountain):** Glaciers and snowy mountain peaks share near-identical visual elements (rock ledges, white snow, horizon sky lines). This resulted in an $8-9\%$ confusion rate between the two classes.
   Solution: Applied Color Jittering and subtle contrast transforms during training to help the model emphasize structural edge features over raw color intensity.
2. **Urban Scene Confusion (Buildings vs. Street):** Street photographs framed by tall architecture often contain substantial building pixel coverage, causing slight misclassification.
   Solution: Replaced default classification heads with Dropout ($p=0.3$) and Adaptive Pooling layers to encourage spatially invariant feature learning.
3. **GPU Memory Limits:** Training large mini-batches with heavy transformations can cause Out-Of-Memory (OOM) errors.
   Solution: Optimized batch size to 64 and enabled pinned memory (`pin_memory=True`) with 2 worker threads in PyTorch DataLoaders.

2.10 Learning Outcomes

- Gained comprehensive expertise in PyTorch dataset pipeline design, multi-stage CNN architecture construction, and Transfer Learning methodologies.
- Understood empirical trade-offs: ResNet-18 required **10.8x fewer trainable parameters** and **35% less training time** while delivering a **+10.5% accuracy gain** over the custom CNN.
- Mastered rigorous evaluation protocols including Confusion Matrices, per-class F1-scores, and qualitative error mode diagnostics.

2.11 Future Scope

- **Grad-CAM Visualization:** Integrating Gradient-weighted Class Activation Mapping (Grad-CAM) to generate heatmap overlays showing exact image regions driving predictions.
- **Model Deployment:** Exporting trained PyTorch models to ONNX runtime and wrapping them inside a web application using Gradio / Streamlit.
- **Hyperparameter Tuning:** Implementing automated hyperparameter search using Ray Tune or Optuna.

CHAPTER 3: SUMMARY AND CONCLUSION

3.1 Overall Learning Experience

The **Summer Training Programme on Data Science, Machine Learning & Agentic AI** organized by the **E&ICT Academy, IIT Roorkee** offered a transformative educational experience. The curriculum harmoniously blended rigorous mathematical foundations with modern engineering practices. Under the guidance of distinguished faculty and industry mentors, participants advanced from fundamental Python data structures to training multi-million parameter neural network models on GPU clusters.

The practical emphasis on end-to-end project building ensured that theoretical knowledge was immediately reinforced through code execution, debugging, model tuning, and quantitative evaluation.

3.2 Skills Acquired

Throughout the programme, the participant mastered a versatile technical skillset:

Competency Domain	Specific Technical Skills Acquired
Data Engineering & EDA	Data cleansing, missing value imputation, outlier detection, data visualization with Seaborn, stratified train-val-test splitting.
Machine Learning Foundations	Supervised algorithms (Linear/Logistic Regression, Decision Trees, Random Forests, XGBoost), Unsupervised clustering (K-Means, PCA).
Deep Learning & PyTorch	Custom Layer construction, AutoGrad, Optimizer selection (AdamW, SGD), Loss Function design, Learning Rate Schedulers, GPU acceleration.
Computer Vision	Convolutional operations, spatial feature maps, transfer learning backbones (ResNet, VGG), dynamic image augmentations.
Model Diagnostics & Evaluation	Confusion Matrices, Precision-Recall curves, F1-score evaluation, bias-variance trade-off analysis, error mode inspection.
Agentic AI & LLM Systems	Prompt engineering, LangChain orchestration, autonomous tool usage, agentic decision loops.

3.3 Key Takeaways

1. **Transfer Learning Dominance:** For medium-scale image datasets, leveraging features pre-trained on massive corpora (ImageNet) delivers vastly superior accuracy, faster convergence, and higher parameter efficiency compared to training networks from scratch.
2. **Importance of Data Augmentation:** Dynamic image transformations (rotation, horizontal flips, color jitter) act as strong regularizers, preventing overfitting and improving generalization on unseen test samples.
3. **Comprehensive Model Diagnostic Skills:** Accuracy alone is insufficient; analyzing per-class recall, precision, and confusion matrices is essential for identifying specific failure modes (e.g., glacier vs. mountain).
4. **Clean Engineering Practices:** Modular code design, seed setting for deterministic reproducibility, and automated logging are vital for production-grade AI systems.

3.4 Applications of the Knowledge Gained

The skills and methodologies developed during this training programme have direct applicability across numerous real-world domains:

- **Remote Sensing & Earth Observation:** Satellite imagery classification for land cover mapping, deforestation tracking, and urban growth monitoring.
- **Autonomous Vehicle Navigation:** Scene understanding, path detection, and obstacle recognition for self-driving vehicles and mobile robotics.
- **Medical Imaging & Healthcare:** Adapting vision pipelines for diagnostic image classification (e.g., X-ray, MRI, CT scan anomaly detection).
- **Smart Agriculture:** Crop health monitoring, weed detection, and yield estimation from aerial drone imagery.

3.5 Conclusion

The Summer Training Programme on Data Science, Machine Learning & Agentic AI organised by the Electronics & ICT Academy, IIT Roorkee successfully achieved all its pedagogical goals. The hands-on project demonstrated the complete lifecycle of a deep learning solution—from data acquisition and preprocessing to custom CNN design, transfer learning integration, and empirical evaluation.

The comparative analysis highlighted the immense power of fine-tuning pre-trained residual backbones (ResNet-18), achieving **93.10% test accuracy** on natural scene classification. The comprehensive knowledge, practical skills, and engineering confidence gained during this

programme lay a robust foundation for pursuing advanced research and industry roles in Data Science, Computer Vision, and Artificial Intelligence.

REFERENCES

The work and technical implementations described in this report drew upon the following foundational research papers, textbooks, official software documentation, and benchmark datasets:

1. **He, K., Zhang, X., Ren, S., & Sun, J. (2016).** Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, pp. 770-778.
2. **LeCun, Y., Bengio, Y., & Hinton, G. (2015).** Deep learning. *Nature*, 521(7553), 436-444.
3. **Goodfellow, I., Bengio, Y., & Courville, A. (2016).** *Deep Learning*. MIT Press. Available online at https://www.deeplearningbook.org/.
4. **Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., et al. (2019).** PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS 32)*, pp. 8024-8035.
5. **Intel Corporation & Kaggle.** Intel Image Classification Dataset. *Kaggle Repository*. Available online at https://www.kaggle.com/datasets/puneet6008/intel-image-classification.
6. **PyTorch Official Documentation.** *Torchvision Models and Transforms*. PyTorch Core Team. Available online at https://pytorch.org/vision/stable/index.html.
7. **Electronics & ICT Academy, IIT Roorkee.** *Summer Training Curriculum and Lecture Notes on Data Science, Machine Learning & Agentic AI* (2026). E&ICT Academy, Indian Institute of Technology Roorkee.
8. **Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., et al. (2015).** ImageNet large scale visual recognition challenge. *International Journal of Computer Vision (IJCV)*, 115(3), 211-252.
9. **Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017).** Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *IEEE International Conference on Computer Vision (ICCV)*, pp. 618-626.
10. **McKinney, W. (2012).** *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*. O'Reilly Media.